PARADIGMAS DE LA PROGRAMACIÓN · UNIDAD 2

El paradigma funcional

Resumen de la clase para preparar el examen

Elaborado a partir de la clase grabada de la UD2, el *Capítulo 2. El paradigma funcional* del temario y la *Guía de Common Lisp* del aula virtual. Las marcas min. 09:42 indican en qué minuto del vídeo se explica cada tema.

Si solo tienes 5 minutos: lo esencial

1. Nace del **cálculo lambda (λ-cálculo) de Alonzo Church, 1934**. Programar es **evaluar expresiones** (funciones matemáticas), no ejecutar órdenes paso a paso.
2. Es **declarativo**: describe **qué** se quiere obtener. El procedimental/imperativo describe **cómo** (modelo de von Neumann).
3. **Transparencia referencial**: con la misma entrada, siempre la misma salida; no hay efectos laterales. Es la base de todo lo demás.
4. **Composición funcional**: el programa es una cadena de funciones; la salida de una es la entrada de otra: `g(f(x))`.
5. **Funciones de orden superior** (funciones como objetos de primera clase): se pasan como argumento y se devuelven como resultado.
6. **No existe el estado**: ni asignaciones destructivas (`a ← b`) ni bucles (`for, while`). Se itera con **recursividad**: caso base + caso recursivo.
7. **Polimorfismo**: de sobrecarga, dinámico (de inclusión) y paramétrico (genericidad). El paramétrico es el más natural en programación funcional.
8. **Evaluación ansiosa** (llamada por valor) frente a **diferida o perezosa** (llamada por necesidad). La diferida permite trabajar con estructuras infinitas.
9. Las **mónadas** (Haskell) encapsulan lo procedimental, como la E/S o el estado, sin perder la pureza.
10. **Ventajas**: simplicidad, razonamiento formal, concisión (10 a 1), abstracción y paralelismo. **Inconvenientes**: poco intuitivo, menos eficiente (de 3 a 5 veces más lento) y poco apropiado para gran escala.

1 Origen y enfoque del paradigma

Orígenes min. 01:52

Es uno de los paradigmas **más antiguos y fundamentales** de la informática. Sus orígenes teóricos son el **cálculo lambda**, un modelo matemático de computación que **Alonzo Church** introdujo en **1934** para sus estudios sobre computabilidad. Su objetivo es una programación **expresiva y matemáticamente elegante**, en la que no haga falta describir paso a paso la secuencia de computación: se basa en la **evaluación de expresiones** en lugar de en la ejecución de comandos.

Especificación procedimental y declarativa de una función

Una misma función se puede especificar de dos formas (pregunta de autocomprobación del tema):

- **Declarativa (matemática)**: un conjunto de **transformaciones disjuntas** (condición → expresión) **sin orden de ejecución**. Ejemplo: `par(x) = { resto(x,2) == 0 → true ; resto(x,2) != 0 → false }`
- **Procedimental**: se indica **el orden** en que se evalúa. Ejemplo: `par(x) = if ((x - (x//2)*2) == 0) then true else false`. Ese orden impuesto se llama *sobrespecificación*.
- **Diferencia clave**: si se especifica o no de forma explícita la **secuencia de ejecución**. Common Lisp admite ambos estilos.

Procedimental frente a declarativo: resumen

	Procedimental (imperativo)	Declarativo (funcional)
Pregunta que responde	**¿Cómo** resolver el problema?	**¿Qué** se quiere conseguir?
Modelo	Máquina de **von Neumann**: un procesador y acceso a memoria	**Función matemática** y composición de funciones
Control	El programador fija la secuencia: `if`, `while`, `for`	Sin orden explícito: transformaciones que se encadenan
Memoria	Variables que se actualizan constantemente (celdas modificables)	Valores que no se modifican
Historia	Dominó al principio porque **aprovechaba el hardware escaso** de la época	Pedía más hardware. Hoy, con multinúcleo y mucha memoria, **se está retomando**

2 Las características (pilares) del paradigma

> **Ojo con la numeración**
> En clase y en la guía de Lisp se habla de **5 pilares**, y el libro enumera **6 características** porque separa el polimorfismo de la evaluación diferida. Los conceptos son los mismos. Apréndete los seis: **transparencia referencial · composición funcional · orden superior · sin estado (recursividad) · polimorfismo · evaluación diferida**.

2.1 Transparencia referencial min. 09:42

Una función tiene transparencia referencial si se comporta como una **función matemática**, como una **caja negra**: si recibe el mismo valor de entrada, **siempre produce el mismo valor de salida** (es determinista). Además, el resultado se devuelve **solo a través del nombre de la función**, no mediante alguno de sus argumentos.

Se pierde cuando hay **efectos laterales** (también llamados efectos de lado o colaterales), por ejemplo al:

- modificar **variables globales** dentro de la función;
- modificar **argumentos pasados por referencia**, como `void cuadrado(int x, int &y)`;
- usar **variables** `static` que conservan su valor entre llamadas: `fun(3)` devuelve 8, luego 10 y luego 12;
- hacer **entrada/salida** o consultar el **hardware** o el estado del sistema.

Ejemplo del libro y de la clase (C++): la misma expresión da resultados distintos

```
int sw;                       // variable global

int fun(int n) {
    int v;
    if (sw) v = n; else v = n * n;
    sw = !sw;                 // efecto lateral: cambia la global
    return v;
}

int main() {
    sw = 0;
    cout << fun(2) + fun(3) << endl;   // 11
    cout << fun(3) + fun(2) << endl;   // 7  -> ¡la suma deja de ser conmutativa!
}
```

Problemas de los efectos laterales: la función deja de devolver siempre lo mismo y los errores son **difíciles de depurar**. **Por qué importa la transparencia**: permite **sustituir iguales por iguales** (razonamiento ecuacional), es **indispensable para la composición** de funciones y facilita **verificar la corrección** de los programas.

En Lisp (guía)

```
;; Impura: depende de un estado externo y lo modifica
(defvar *acumulador* 0)
(defun acumular-impuro (x)
  (setf *acumulador* (+ *acumulador* x)))

;; Pura: misma entrada -> misma salida, sin efectos laterales
(defun sumar-puro (a b)
  (+ a b))
```

2.2 Composición funcional min. 14:26

El programa no se ve como una lista de tareas, sino como una **función compuesta de otras más simples**: el resultado de una función es el argumento de entrada de la siguiente. Si `y = f(x)` y `z = g(y)`, entonces `z = g(f(x))`.
Requisito: transparencia referencial.

Ejemplo de clase: dada una lista, una función **selecciona los pares** y su resultado pasa a otra que **los suma**. Es la forma típica de manipular listas (ordenar, filtrar, operar sobre cada elemento).

En Lisp (guía)

```
;; Elevar al cuadrado y sumar: la salida de mapcar alimenta a reduce
(defun suma-cuadrados (lista)
  (reduce #'+ (mapcar (lambda (x) (* x x)) lista)))

(print (suma-cuadrados '(2 3 4)))   ; 4 + 9 + 16 = 29
```

En Hope (libro)

```
dec sumarnumeros : num # num -> num;
--- sumarnumeros(x, y) <= suma(cuadrado(mayor(x, y)));
! sumarnumeros(12, 5): mayor = 12, cuadrado = 144, suma 1+2+...+144 = 10440
```

2.3 Funciones de orden superior (objetos de primera clase) min. 16:23

- Las funciones se tratan **como cualquier otro dato**: se pasan como parámetros, se devuelven como resultado, se asignan y se mezclan con otros datos.
- Una **función de primer orden** no recibe ni devuelve funciones. Una **función de orden superior** sí (se llama también de primera clase o funcional).
- En C se simula con **punteros a función**. En los lenguajes funcionales es **nativo** y da un código **reutilizable y abstracto**. Está muy ligado a la composición.

En Hope (libro)

```
dec calcularrec : num # num # (num # num -> num) -> num;
--- calcularrec(x, y, f) <= f(x, y);
! calcularrec(3, 4, area) = 12      calcularrec(3, 4, perimetro) = 14
```

En Lisp (guía)

```
(defun aplicar-operacion (fn a b)
  (funcall fn a b))                          ; llama a la función recibida

(print (aplicar-operacion #'+ 10 5))         ; 15
(print (apply #'+ '(1 2 3 4 5)))             ; 15  (argumentos en una lista)
(print (mapcar (lambda (x) (* x x)) '(1 2 3 4))) ; (1 4 9 16)  aplica a cada elemento
(print (reduce #'+ '(1 4 9 16)))             ; 30  combina hasta un solo valor
```

> **Pregunta trampa**
>
> **¿Es válido usar un puntero en el paradigma funcional?** No, porque un puntero incorpora el concepto de **estado de memoria**, que en este paradigma no existe.

2.4 No existe el estado: recursividad en lugar de bucles min. 18:11

No existe la **celda de memoria modificable**. Los resultados intermedios se guardan como **valores locales nuevos** y el flujo siempre avanza. En consecuencia:

- **no hay asignaciones destructivas**: `a ← b`, `x = x + 1`, `i++`;
- **no hay procesos iterativos** (`for`, `while`, `do`), porque necesitan mutar una variable de control y un acumulador.

La solución es la recursividad min. 20:00: la función se llama a sí misma con un problema más pequeño (**caso general o recursivo**) hasta llegar a un valor conocido que la detiene (**caso base**). Sin caso base, la recursión sería infinita.

Factorial: iterativo frente a recursivo

```cpp
// Imperativo (C++)
long factorial(int x) {
    long total = 1;
    while (x > 0) {     // bucle
        total *= x;     // asignación destructiva
        x--;            // mutación
    }
    return total;
}
```

```lisp
;; Funcional (Lisp)
(defun factorial (n)
  (if (<= n 0)
      1                          ; caso base
      (* n (factorial (- n 1)))) ; recursivo

;; En Hope (libro):
;; factorial(x) <= if x =< 0 then 1
;;                 else x * factorial(x-1);
```

Patrón con acumulador (guía): el contador viaja como parámetro opcional

```lisp
;; Primer número par y su posición (desde 1), sin mutar ningún contador
(defun primer-par-pos (lista &optional (pos 1))
  (cond
    ((null lista) nil)                          ; caso base: no hay pares
    ((evenp (car lista)) (list (car lista) pos))  ; éxito: (valor posición)
    (t (primer-par-pos (cdr lista) (+ pos 1)))))  ; recursión sobre la cola

(print (primer-par-pos '(1 3 8 7)))   ; (8 3)
```

2.5 Polimorfismo min. 22:45

Es la capacidad de una función u operador de trabajar con **distintos tipos de datos**. Los lenguajes **monomórficos**, como Pascal, asignan un único tipo a cada valor. Hay tres tipos de polimorfismo:

Tipo	Idea	Ejemplo
Sobrecarga (*ad hoc*, Strachey)	Mismo nombre, varias versiones según el número o el tipo de los argumentos	`imprimir(int)`, `imprimir(char)`, `imprimir(int, float)`
Dinámico o de inclusión	El método se decide en ejecución según la **clase** del objeto (jerarquías de clases)	*Empleado* es subclase de *Persona* y se usa donde se espera una Persona
Paramétrico o genericidad	**Una sola definición** (plantilla) que sirve para cualquier tipo	`template <class T> void imprimir(T x)`. En Hope, con `alpha`

El **polimorfismo paramétrico es natural en el paradigma funcional** y resulta más forzado en el procedimental. Ejemplo de clase: una función que trabaja con listas sirve para listas de enteros, de decimales o de palabras.

En Lisp (guía)

```lisp
(defun tomar-n (n lista)                    ; primeros n elementos de CUALQUIER lista
  (cond
    ((or (<= n 0) (null lista)) nil)
    (t (cons (car lista) (tomar-n (- n 1) (cdr lista))))))

(print (tomar-n 2 '(10 20 30 40)))            ; (10 20)
(print (tomar-n 2 '("alfa" "beta" "gamma")))  ; ("alfa" "beta")
```

2.6 Evaluación ansiosa frente a diferida min. 24:23

	Ansiosa (*eager*)	Diferida o perezosa (*lazy*)
Cuándo evalúa los argumentos	**Antes** de llamar a la función	Solo **cuando el cuerpo los necesita**
Tipo de llamada	**Por valor** (*call by value*)	**Por necesidad** (*call by need*)
Orden	Evaluación de 1.er orden	Evaluación de 2.º orden
Ventaja	Fácil de implementar, eficiente y rápida	Permite **estructuras infinitas** y separa el control del cálculo
Inconveniente	Evalúa argumentos que quizá no se usen	Implementación más cara (pasa expresiones sin evaluar)
Lenguajes	Lisp, ML y la mayoría de los imperativos	**Haskell**, Lazy ML y Hope (una de sus implementaciones)

Lista infinita (Hope, libro). En clase: «una lista infinita de pares de la que solo uso los 3 primeros»

```
dec lista : num -> list(num);
--- lista(n) <= n :: lista(n + 1);          ! lista infinita: n, n+1, n+2, ...

dec primerimpar : list(num) -> num;
--- primerimpar(nil)    <= 0;
--- primerimpar(x :: l) <= if x mod 2 /= 0 then x else primerimpar(l);

! primerimpar(lista(8)):  ansiosa -> intenta construir la lista entera: NO TERMINA
!                         diferida -> 8 es par, pide el siguiente -> 9
```

Por qué la evaluación diferida encaja mal con el paradigma procedimental: el programador pierde el control del orden de evaluación y, si hay efectos laterales, el resultado cambia. Con `cuadrado(getint(f))` y un fichero que contiene 2, 5, …, la evaluación ansiosa da `2*2 = 4`, pero la diferida da `getint(f) * getint(f) = 2*5 = 10`.

3 Mónadas

El problema de la entrada/salida min. 26:36

Interactuar con la consola, el teclado o la pantalla produce efectos laterales. Una **mónada** es un **elemento encapsulado que proporciona características procedimentales sin perder los beneficios del paradigma funcional**: la transparencia referencial, el orden superior y la evaluación diferida. Así el programa sigue siendo puro y además permite un desarrollo **estructurado y secuencial**.

- Aparecen en **Haskell**. Se usan para la **E/S** (la **mónada IO**), el estado mutable (por ejemplo, en un parser) y las excepciones.
- Su estructura es un **constructor de tipo unario** `m` y dos funciones: `return :: a -> m a`, que mete un valor en la mónada, y `>>= :: m a -> (a -> m b) -> m b` (*bind*), que encadena acciones pasando el resultado de una a la siguiente. Deben cumplir las **3 leyes monádicas**.
- Son necesarias en Haskell porque, con evaluación diferida, no se sabe en qué orden se ejecutarán las llamadas: `>>=` impone la secuencia.

Mónada IO en Haskell (libro)

```
exIO :: IO ()
exIO = getChar    >>= \c1 ->
       getChar    >>= \c2 ->
       putChar c2 >>= \_  ->
       putChar c1              -- lee dos caracteres y los escribe en orden inverso
```

> **Pregunta trampa**
>
> **¿Son las mónadas una característica básica de la programación funcional?** No. Son elementos encapsulados que **añaden** características procedimentales, no uno de los pilares del paradigma.

4 Programa funcional, tipos de datos y lenguajes

Un programa funcional es *«un conjunto compuesto de funciones que juntas definen un objetivo específico de aplicación»*. Se resume en la ecuación **Funciones + Estructuras de datos = Programa**.

- **Elementos fundamentales**: las **funciones**, la **recursividad** (iteración con asignación no destructiva) y las **listas**.
- **Tipos de datos básicos**: **átomos** (`28`, `43.76`, `T`, `NIL`, `'h'`), **identificadores** (nombres de variables y funciones) y **listas**. Una cadena es una lista de caracteres: `"hola"` = `('h' 'o' 'l' 'a')`. Se accede a la **cabeza** (primer elemento) y a la **cola** (el resto).
- **Currificación**: una función de n argumentos se trata como una cadena de n funciones de un argumento: `f x y z = (((f x) y) z)`.
- Los lenguajes funcionales son **aplicativos** (aplican funciones de forma progresiva). Los primeros eran **interpretados**; hoy suelen ser compilados.

Lenguajes vistos en la asignatura min. 29:00

Lenguaje	Lo que hay que saber
Lisp	**1959, John McCarthy, MIT** (*List Processing*). Es el **primero que implementa la programación funcional**, pero es **híbrido** (funcional y procedimental): no nació funcional y el paradigma se le añadió después. Tiene `setq`/`setf`, `loop`, arrays y variables globales. **No tiene evaluación diferida.** Su estándar es **Common Lisp**.
Hope	Nació para el paradigma funcional, en un **entorno académico**. Tiene pocas aplicaciones reales y no hay intérpretes en línea: hay que instalar un binario. Es útil para ver fundamentos que Lisp no tiene.
Haskell	Nació para el paradigma funcional y se usa **fuera del ámbito académico**. Tiene **evaluación diferida** y **mónadas**. Es el modelo de la programación funcional moderna.
Otros	Python, Java, C#, Scala, Kotlin, F#, Rust… soportan varios paradigmas. Valen para la AEC solo si se programa con **enfoque funcional**, sin mutar variables.

5 Ventajas e inconvenientes frente al imperativo min. 33:00

Ventajas	Inconvenientes
Simplicidad: menos comandos. Haskell tiene 23 identificadores y un manual de 80 páginas; Ada, 62 identificadores y 200 páginas.	**Dificultad**: es **poco intuitivo**, porque estamos acostumbrados a pensar de forma procedimental y con bucles. Se consigue con práctica.
Razonamiento formal: la transparencia referencial permite el razonamiento **ecuacional** (sustituir iguales por iguales).	**Ineficiencia**: es de **3 a 5 veces más lento** y la recursividad consume más memoria.
Concisión: hasta **10 a 1** en líneas de código. Menos código implica más fiabilidad.	**Gran escala**: no es apropiado para la programación a gran escala, aunque sí para investigar. Haskell, Standard ML y Miranda están mejorando.
Abstracción: se ajusta al modelo del problema, no al del computador. El código es más reutilizable.	
Paralelismo: sin efectos laterales es **ideal para la programación paralela y concurrente** (multinúcleo, hilos).	

6 Common Lisp: chuleta de lo visto en clase

Para probar el código sin instalar nada se puede usar un intérprete de Common Lisp en línea, como el de la clase. Todo se escribe como **S-expresiones**: entre paréntesis y en **notación prefija** o polaca, con el operador primero: `(operador arg1 arg2 ...)` min. 43:29

Concepto	Sintaxis	Ejemplo → resultado
Operación	`(op a b)`	`(+ 2 3)` → 5; `(* 2 (+ 4 5))` → 18
No evaluar (dato literal)	`'x` o `(quote x)`	`'(1 2 3)` → (1 2 3)
Imprimir	`(print x)`	`(print '(+ 2 3))` → (+ 2 3), sin calcular
Imprimir con formato	`(format t "~A~%" x)`	`t` = terminal, `~A` = valor legible, `~%` = salto de línea
Comentarios	`; ;; ;;;`	corto · línea completa · título de bloque. **No** es fin de sentencia como en C
Variable global	`(defparameter *x* 10)`	Convención de asteriscos (*earmuffs*). También `defvar`
Variable local	`(let ((a 1) (b 2)) cuerpo)`	Solo existe dentro de los paréntesis del `let`
Función	`(defun nombre (params) cuerpo)`	`(defun sumar (a b) (+ a b))`
Parámetro opcional	`&optional (p valor)`	Útil para acumuladores y contadores
Condicional simple	`(if test si-no)`	Una condición, una rama verdadera y una falsa
Condicional múltiple	`(cond (t1 e1) (t2 e2) (t eN))`	Varias ramas en orden. `(t ...)` = else
Función anónima	`(lambda (params) cuerpo)`	`((lambda (b h) (* b h)) 10 5)` → 50
Construir listas	`(cons x l)`, `(list a b c)`	`(cons 5 '(10 20))` → (5 10 20)
Cabeza / cola	`(car l)`, `(cdr l)`	`(car '(10 20 30))` → 10; `cdr` → (20 30)
Lista vacía	`(null l)`	`T` si está vacía y `NIL` si no: sirve de **caso base**
Par / impar	`(evenp n)`, `(oddp n)`	`(evenp 4)` → T
Función como dato	`#'f`, `funcall`, `apply`	`(funcall #'+ 10 5)` → 15; `(apply #'+ '(1 2 3))` → 6
Map / reduce	`(mapcar f l)`, `(reduce f l)`	`(mapcar (lambda (x) (* 2 x)) '(1 2))` → (2 4)

if y cond (ejemplos de la guía, min. 59:14)

```
(defparameter *edad* 20)
(if (>= *edad* 18)
    (format t "Eres mayor de edad.~%")     ; si es verdadero
    (format t "Eres menor de edad.~%"))    ; si es falso (nil)

(defparameter *nota* 15)
(cond
  ((>= *nota* 19) (format t "Excelente~%"))
  ((>= *nota* 14) (format t "Aprobado~%"))      ; <- se imprime este
  ((>= *nota* 11) (format t "Suficiente~%"))
  (t              (format t "Reprobado~%")))    ; t = rama por defecto (else)
```

let, defun y lambda (min. 51:22, 56:01 y 1:06:21)

```
(print (let ((base 10)
             (altura 5))
         (/ (* base altura) 2)))          ; 25. base y altura no existen fuera del let

(defun saludar-usuario (usuario)
  (print (concatenate 'string "hola " usuario ", como estas?")))
(saludar-usuario "Antonio")              ; "hola Antonio, como estas?"

(print ((lambda (base altura) (* base altura)) 10 5))   ; 50, función anónima
```

Listas y recursión: el patrón que más vas a usar (ejemplo 2.19 del libro en Common Lisp actual)

```
;; Lista '(10 20 30) en memoria: cada celda cons tiene car (dato) y cdr (resto)
;;   [10 | •]→[20 | •]→[30 | NIL]

(defun sumar-lista (l)
  (if (null l)
      0                                   ; caso base: lista vacía
      (+ (car l) (sumar-lista (cdr l)))))  ; cabeza + suma de la cola

(print (sumar-lista '(4 8 5 10)))         ; 27
```

Errores típicos en Lisp

- Olvidar que **todo va entre paréntesis** y que el **operador va primero**: `(>= edad 18)`, no `(edad >= 18)`.
- Olvidar el **apóstrofe** en las listas de datos: `(car '(1 2 3))`. Sin él, Lisp intenta ejecutar `1` como si fuera una función.
- La `t` de `format` indica la salida por terminal; la `t` final de `cond` hace de else.
- En el intérprete en línea de la clase, `print` no muestra bien las **tildes** ni el **¿**.
- Lisp permite `setf`/`setq`, `loop` y variables globales porque **no nació funcional**. Si los usas, pierdes el enfoque funcional (y puntos en la AEC).

Concepto	Imperativo (C / Java)	Funcional (Common Lisp)
Flujo de ejecución	Sentencias secuenciales, `for`, `while`	Evaluación de expresiones recursivas
Variables	Celdas de memoria mutables (`i++`)	Ligaduras inmutables (`let`, argumentos)
Funciones	Subrutinas con posibles efectos laterales	Funciones puras (caja negra matemática)
Recorrer colecciones	Índices: `array[i]`	Cabeza y cola: `car` / `cdr`
Transformar en lote	Bucles con acumuladores manuales	`mapcar` y `reduce`

7 Preguntas tipo examen (con respuesta)

1. **¿De qué dos formas se suele especificar una función?**
   Declarativamente (matemáticamente) o procedimentalmente.
2. **¿Cómo se llama la propiedad por la que una función, con los mismos datos de entrada, solo puede producir un único valor de salida?**
   Transparencia referencial.
3. **¿Qué caracteriza a una función de orden superior?**
   Que sus argumentos y su valor de retorno pueden ser otras funciones.
4. **¿Sería válido utilizar un puntero en este paradigma?**
   No, porque un puntero incorpora el concepto de estado de memoria, que no es válido en el paradigma.

5. **¿Qué es una función con polimorfismo paramétrico?**
   La que implementa una misma ejecución admitiendo argumentos de distintos tipos de datos.
6. **¿Qué tipo de llamada se realiza en la evaluación ansiosa y en la diferida?**
   En la ansiosa, una llamada por valor (*call by value*); en la diferida, una llamada por necesidad (*call by need*).
7. **¿Qué es una mónada?**
   Un elemento encapsulado que proporciona características procedimentales sin perder los beneficios del paradigma funcional.
8. **¿Cuáles son los elementos fundamentales de un programa funcional?**
   Las funciones, la recursividad y las listas.
9. **¿Son las mónadas una característica básica de la programación funcional?**
   No. Son elementos encapsulados que aportan características procedimentales.
10. **Enumera las ventajas del paradigma funcional frente al imperativo.**
    Simplicidad, posibilidad de razonamiento formal, concisión, abstracción y paralelismo.
11. **¿Qué es un efecto lateral? Pon ejemplos.**
    Que una función modifique o dependa de algo externo a ella, de modo que la misma llamada dé resultados distintos. Ejemplos: variables globales, parámetros por referencia, variables `static` y operaciones de E/S.
12. **Si no hay bucles, ¿cómo se itera?**
    Con recursividad: un caso base que detiene la recursión y un caso recursivo que reduce el problema (en listas, avanzando con `cdr`).
13. **¿Por qué la evaluación diferida encaja mal en el paradigma procedimental?**
    Porque el programador pierde el control del orden de evaluación y, con efectos laterales, los resultados cambian (`cuadrado(getint(f))`: 4 con evaluación ansiosa y 10 con diferida).
14. **¿Es Lisp un lenguaje funcional puro?**
    No. Es híbrido: tiene asignación destructiva (`setq`), bucles (`loop`), arrays y variables globales, y no tiene evaluación diferida.
15. **¿Qué diferencia hay entre `if` y `cond` en Lisp?**
    `if` evalúa una sola condición con una rama verdadera y otra falsa. `cond` evalúa varias ramas en orden, y `(t ...)` actúa como else.
16. **¿Por qué Hadoop o Spark usan `map` y `reduce` en vez de un bucle `for`? (AEC1, ejercicio 10)**
    `map` aplica una función pura a cada elemento de forma independiente y sin estado compartido, así que el trabajo se puede repartir entre muchos nodos en paralelo. `reduce` combina después los resultados parciales. Sin efectos laterales no hay conflictos entre los nodos.
17. **¿Por qué el paradigma funcional vuelve a estar de moda?**
    Porque las limitaciones de hardware que lo frenaron (poca memoria y un solo procesador) ya no existen, y la ausencia de efectos laterales facilita aprovechar los procesadores multinúcleo y la concurrencia.

8 Relación con la AEC1

En clase se presentó la AEC1 sobre el paradigma funcional min. 38:09. Datos clave del enunciado:

- Se puede hacer en Lisp, Hope, Haskell… o en Python, Java, etc., **siempre que el enfoque sea funcional**, usando los pilares y sin mutar variables.
- Se entrega **un fichero de código** (`.lsp`, `.hop`, `.hs`…) llamado `nombre_apellido1_apellido2_AEC1.lsp`. Debe llevar una cabecera en comentarios (`; Paradigmas de la Programación`, `; AEC1`, `; Nombre: ...`), cada ejercicio marcado (`; Ejercicio 2`) y las **respuestas teóricas como comentarios**.

- Se valoran la explicación de las soluciones, que sean correctas, la entrega en plazo y con el formato pedido, y usar **exactamente los nombres de función del enunciado**.

Ejercicio	Concepto que pone a prueba
1 imprimir_bienvenida	Salida por pantalla (`format` / `print`)
2 longitud_lista	Recursividad pura sin `length`: caso base con `null` y avance con `cdr`
3-4 encontrar_minimo (y posición)	Recursión sobre listas; posición con acumulador `&optional`, como `primer-par-pos`
5 sumar_max_y_min	Reutilizar y generalizar funciones: orden superior (pasar `#'<` o `#'>`)
6-7 obtener_caracter_n / trinomio_n	Recorrer con contador y **polimorfismo paramétrico** (¿se puede generalizar?)
8 duplicar_lista	Recursividad y `mapcar` + `lambda`. Qué es una lambda y sus ventajas
9 repetir_accion_n	Función como parámetro (`funcall`) y recursión en lugar de bucles
10 map/reduce y tipado	Teoría: paralelismo sin efectos laterales y tipado del lenguaje (Lisp: dinámico)

9 Mapa de la clase (para buscar en el vídeo)

Minuto	Tema	Minuto	Tema
00:26	Presentación de la UD2: el paradigma funcional	**34:56**	Inconvenientes y dificultad de pensar en funcional
01:52	Orígenes (Church, λ-cálculo) y procedimental frente a declarativo	**37:51**	Material del aula virtual y AEC1
06:30	Los pilares del paradigma	**39:41**	Entorno en línea para probar Lisp
09:42	Transparencia referencial y efectos laterales (ejemplo en C++)	**43:29**	S-expresiones y notación polaca
14:26	Composición funcional	**45:24**	print, apóstrofe y quote
16:23	Funciones como objetos de primera clase	**48:15**	Operaciones anidadas y comentarios
18:11	Ausencia de estado	**51:22**	Variables globales y locales (let)
20:00	Recursividad en lugar de bucles	**56:01**	Definir funciones con defun
22:45	Polimorfismo: sobrecarga, dinámico y paramétrico	**59:14**	Condicionales: if y cond
24:23	Evaluación ansiosa y diferida; listas infinitas	**1:06:21**	Funciones lambda
26:36	Mónadas	**1:09:39**	Listas: cons y list
29:00	Lenguajes: Lisp, Hope y Haskell	**1:10:40**	car, cdr, null y evenp/oddp
33:00	Ventajas del paradigma	**1:12:44**	Recomendaciones; la semana que viene, los mismos ejemplos en Hope y Haskell